A LEARN-BY-DOING COMPANION

From booking data to business decisions.

A step-by-step explanation of your hotel cancellation project, with small datasets, worked calculations, SQL, Python, exercises, and honest resume language.

Beginner friendly | SQL first | Six project phases | Practical examples | Print ready

Document status: educational guide, not a completed analysis. No Kaggle data has been loaded, no project model has been trained, and no dataset results are claimed.

01 / START HERE

What you are getting, and how to read it

Your original prompt is a specification: it tells an assistant what to build, in what order, and what standards to follow. This guide explains that specification. It does **not** silently execute all six phases or skip your review checkpoints.

> **Your implementation uses PostgreSQL + DBeaver.** PostgreSQL is the database engine; DBeaver is your visual SQL client. Starter code for all phases is prepared, but your real-data work is not executed. Follow README.md for the project overview and reproduction steps. The final chapter of this PDF includes manual commands and review gates. No commit or push is performed automatically; your recommendation content remains yours.

> **Numbers policy.** Every numeric booking example or model score below is explicitly fictional teaching material. Actual project findings must use placeholders such as `[RESULT: overall cancellation rate]` until computed and verified. Dates, code constants, phase numbers, and exercise inputs are not claimed findings.

A useful way to study

1. Read the business question before looking at code.
2. Work through the tiny eight-booking example by hand.
3. Read the SQL and explain each line aloud.
4. Try the exercise before opening its answer.
5. At the end of each phase, use its review checklist before moving on.

Examples versus implementation: code snippets explain individual ideas. Unless labeled self-contained, they assume the tables or variables described nearby. They are not a complete production pipeline. The proposed file layout is an example, not a set of files already created.

Your role

Download the dataset, review each phase, decide how you want the dashboard to look, and write the recommendations yourself.

The assistant's role

Implement the agreed phase, explain decisions, document assumptions, make results reproducible, and stop for your review.

What success looks like: you can explain what each output means, how it was produced, and what it cannot prove. That is more valuable than a repository full of code you cannot defend in an interview.

02 / BUSINESS FRAMING

Start with a decision, not a model

A cancellation affects planning. A hotel may hold a room for someone who later cancels, while demand from other guests changes. An operations team needs visibility into likely cancellations, but it must also avoid treating reliable guests unfairly.

Three different questions

Question	Kind of work	What it can tell you
Which groups canceled more often in the historical data?	Descriptive analysis	Past patterns and group comparisons.
How likely is this booking to cancel?	Predictive modeling	An estimated probability, subject to validation.
Will sending a reminder prevent cancellation?	Causal evaluation	Requires an experiment or an appropriate causal design; SQL and a risk model alone cannot answer it.

> **Practical scenario:** a model flags a booking as higher risk. A sensible next step might be a low-cost confirmation reminder tested in a controlled pilot. It is not automatically a reason to cancel the guest's reservation, charge them more, or overbook the hotel.

Define when the prediction is made

Write a sentence such as: **"Estimate cancellation risk using information available when the booking is first created."** This sentence determines which columns are allowed. A detail learned after the booking was made cannot be used to simulate a booking-time prediction.

Alternatively, predicting a week before arrival is a different product with different allowable features. Do not mix the two silently.

Turn the problem into a project question

"*How do booking characteristics relate to cancellation, and can an interpretable model identify higher-risk bookings in a later time period?*"

Try it: a segment has a higher cancellation rate and often uses a particular channel. Can you conclude that the channel causes cancellations?

Show the explanation

No. That channel may attract bookings made much further in advance, different customer types, or different hotels. Those differences can explain part of the pattern. You can report an association and propose a test, not claim causation.

03 / READ THE DATA LIKE A BOOKING

What a row and a column mean

A **row** represents one booking record, not necessarily one guest, one night, or one unique person. A **column** stores an attribute. Your analysis must keep that unit of measurement consistent.

Column	Plain English	Practical interpretation
`is_canceled`	The target outcome.	1 means canceled; 0 means not canceled. "Not canceled" should not automatically be relabeled "completed stay" without checking status definitions.
`hotel`	The hotel category.	For example, City Hotel or Resort Hotel.
`lead_time`	Days between entering the booking and scheduled arrival.	A lead time of 120 is an advance booking; 5 is much closer to arrival.
`market_segment`	The commercial category.	Online TA describes an online travel-agent segment; it is not the same field as distribution channel.
`distribution_channel`	The route used to distribute the booking.	TA/TO refers to travel agents / tour operators; Direct is a separate route.
`deposit_type`	The recorded deposit arrangement.	Treat No Deposit, Non Refund, and Refundable as categories; do not assume one numeric ordering.
`customer_type`	The booking's customer classification.	Examples include Transient, Transient-Party, Contract, and Group. Verify definitions in the data dictionary.
`is_repeated_guest`	Repeat-guest indicator.	1 means flagged as a repeated guest, not a count of previous stays.
`total_of_special_requests`	A count of recorded requests.	A request might signal engagement, but it is not proof of intent or a causal protection against cancellation.
`adr`	Average daily rate.	A rate, not total booking revenue. Its timing and updates must be checked before using it as a booking-time feature.
`reservation_status`, `reservation_status_date`	Final status and the date of that status.	Outcome-related information that must not enter the cancellation model.

Our shared miniature dataset

FICTIONAL TEACHING DATA - NOT ROWS FROM KAGGLE

The IDs below exist only to make the explanation easy to follow. They are not a promise that the source CSV has a reliable booking ID. "None" abbreviates No Deposit; "Nonrefund" abbreviates Non Refund.

Eight invented bookings used in the SQL and segmentation chapters.

ID	Hotel	Lead days	Channel	Deposit	Repeat	Requests	Canceled
A	City	10	Direct	None	1	1	0

ID	Hotel	Lead days	Channel	Deposit	Repeat	Requests	Canceled
B	City	120	TA/TO	None	0	0	1
C	Resort	90	TA/TO	None	0	2	0
D	Resort	5	Direct	Nonrefund	0	0	0
E	City	200	TA/TO	None	0	0	1
F	City	40	Corporate	None	1	0	0
G	Resort	60	Direct	Refundable	0	1	1
H	Resort	15	TA/TO	None	0	0	0

Work it out: B, E, and G canceled. That is 3 canceled bookings out of 8 total: a fictional cancellation rate of **37.5%**. It is a booking rate, not the percentage of guests or room nights canceled.

04 / TOOLS AND SETUP

Give each tool a clear job

Tool	Job in this project	Simple analogy
CSV	The downloaded source file.	A flat spreadsheet saved as text.
PostgreSQL	Store/query the booking data and execute analytical SQL.	Your existing database engine.
DBeaver	Connect to PostgreSQL, inspect tables, and run SQL files.	A visual workbench, not a second database.
SQL	Filter, group, count, calculate rates, and export summaries.	Instructions for asking structured questions of tables.
Python	Run the workflow, build models, and orchestrate outputs.	The coordinator between the tools.
pandas / NumPy	Prepare PostgreSQL model input for scikit-learn.	A modeling bridge, not the replacement for SQL analysis.
scikit-learn	Preprocess features, fit models, and calculate evaluation metrics.	A machine-learning toolkit.
pytest	Automatically check agreed data-quality rules.	A repeatable inspection checklist.
Git	Track focused changes and their reasons.	A versioned project history.
Power BI / Tableau	Visualize the exported summaries.	The presentation layer you build yourself.

Suggested project layout

```
hotel-cancellation-project
|-- data
|   |-- raw                 Original Kaggle CSV; unchanged
|   |-- processed           Local SQL summaries and model run bundles
|   `-- exports             Dashboard-ready CSVs
|-- sql                     One business question per .sql file
|-- src                     Python workflow and modeling modules
|-- tests                   Data-quality and transformation checks
|-- DATA.md                 Cleaning rules and dataset definitions
|-- README.md               Project story and reproduction commands
|-- recommendation.md       Your headings-only recommendation template
|-- requirements.txt        Direct Python dependencies
`-- .gitignore              Exclude environments and generated/raw data
```

Keep raw booking files and local databases out of Git by default. Record where to download the dataset instead. Review licensing before redistributing any data or derived extracts.

What setup commands would look like on Windows

```
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
.\.venv\Scripts\python.exe -m pytest
```

A virtual environment isolates this project's packages. Using its Python executable directly also avoids relying on shell activation. These commands assume a requirements file and tests have been created during implementation.

The requirements file lists `psycopg`, `python-dotenv`, `numpy`, `pandas`, `scikit-learn`, and `pytest`. It pins the tested direct dependencies. PostgreSQL and DBeaver are installed separately; DuckDB is not required.

> **Why SQL first?** Someone reviewing the repository should find the actual business calculations in named SQL files, not have to search through Python notebook cells to discover how a rate was calculated.

05 / PHASE 1

Setup and data quality: make the denominator trustworthy

Cleaning is not "delete anything unusual." It is a series of documented decisions about meaning, validity, and scope. Preserve the source and build a clean analytical layer separately.

Step 1: Load without hiding parsing problems

The importer reads your local CSV into PostgreSQL using text columns first. This preserves unexpected values for inspection before typed cleaning. Reading a raw table successfully is not the same as validating it.

```
# PowerShell, from the project folder after configuring .env:
.\.venv\Scripts\python.exe -m src.load_data

# Then inspect the schema with DBeaver.
```

Python uses `COPY FROM STDIN`, so the PostgreSQL server does not need access to a Windows file path on your machine. Do not separately use DBeaver's import wizard: use the importer for consistent source record numbers, checksums, and quality auditing. See DATA.md for the exact null markers.

Step 2: Ask what missing really means

Example problem	Tempting mistake	Better documented decision
`children` is blank.	Always replace it with 0.	Missing is unknown, not necessarily no children. Profile it, preserve a flag, and justify any imputation separately.
`agent` is blank.	Delete the booking.	Check whether this represents no agent or unknown agent. Retain the booking unless there is a justified exclusion.
`country` is missing.	Guess it from hotel location.	Keep it unknown; the guest's country is not the hotel's country.
`is_canceled` is not 0 or 1.	Convert every unexpected value to 0.	Flag the invalid label and exclude/quarantine it from labeled analysis under an explicit policy.

Step 3: Handle zero-guest records carefully

FICTIONAL CLEANING EXAMPLES

A row with adults = 0, children = 0, and babies = 0 has no recorded guests. A row with adults = 0 and children = 2 does not have zero guests. A row with children missing has an unknown total, not a confirmed total of zero.

```
-- Assumes guest columns have already been validated and typed.
SELECT
```

```
    *,
    CASE
        WHEN adults IS NULL OR children IS NULL OR babies IS NULL
            THEN 'unknown_guest_count'
        WHEN adults = 0 AND children = 0 AND babies = 0
            THEN 'zero_guests'
        ELSE 'positive_guests'
    END AS guest_count_status
FROM hotel_booking.bookings_quality;
```

For a guest-booking analysis, confirmed zero-guest records may be excluded from the clean analytical table while remaining in raw data. Do not apply that exclusion before inspecting the records and documenting the choice.

Step 4: Construct and validate dates

FICTIONAL DATE CONSTRUCTION EXAMPLE

Year = 2017, month = July, day = 15 becomes a single arrival date: `2017-07-15`. A constructed date such as February 31 is invalid.

```
-- Fictional PostgreSQL example with known valid pieces:
SELECT make_date(2017, 7, 15) AS arrival_date;

-- Review actual parse failures after running the importer:
SELECT source_record_number, exclusion_reasons
FROM hotel_booking.bookings_quality
WHERE 'invalid_arrival_date' = ANY(exclusion_reasons);
```

`make_date` constructs a calendar date. The actual cleaning SQL first maps English month names and uses a session-local safe wrapper: impossible dates become NULL with an explicit exclusion reason. It never silently normalizes February 31 into another date. Inspect the flagged raw values before approving the policy.

Step 5: Investigate duplicates and implausible values

Identical rows are not automatically duplicate bookings. Two different guests can make bookings with identical recorded attributes. Without a trusted identifier, preserve them by default or justify another policy; report any sensitivity analysis separately.

Negative lead times, negative guest counts, invalid labels, and failed numeric casts need flags. Very high room rates or zero-night stays deserve investigation, not arbitrary deletion. A rare value is not necessarily a wrong value.

Step 6: Keep a cleaning ledger in DATA.md

Issue	Decision	Reason	Count
Confirmed zero guests	Exclude from the guest-booking analytical layer after review.	Outside the defined analysis population.	[RESULT: zero-guest exclusions]
Identical records without a reliable ID	Retain by default; document the audit.	Cannot prove they are duplicate bookings.	[RESULT: identical-row count]

Issue	Decision	Reason	Count
Invalid arrival date	Quarantine from time-based outputs; review source values.	Cannot assign a trustworthy period.	[RESULT: invalid-date count]
Missing guest counts	Flag as unknown; define treatment for each output.	Unknown is not zero.	[RESULT: unknown-guest count]

Reconciliation: raw rows = retained analytical rows + excluded rows, when those categories are mutually exclusive. If one record has several issue flags, do not add overlapping issue counts and call that the number of excluded rows.

Try it: you find a booking with adults = 0, children = NULL, babies = 0. Should it count as a confirmed zero-guest booking?

Show the explanation

No. The children count is unknown. Replacing NULL with zero before applying the exclusion would create an unsupported assumption and could delete a real booking.

Stop and review: approve the population definition, missing-value rules, date handling, duplicate policy, exclusion counts, and automated checks before Phase 2.

06 / PHASE 2

SQL analysis: ask one clear question at a time

Most cancellation queries share the same structure: select a population, group if needed, count bookings, count cancellations, and divide using that same population.

Cancellation rate = canceled bookings / all eligible bookings

Run the eight-booking example yourself

The following SQL is self-contained in a PostgreSQL session in DBeaver and creates only fictional teaching data. Its short hotel/deposit labels are simplified for readability. Temporary tables exist only in the connection that creates them.

```
DROP TABLE IF EXISTS pg_temp.toy_bookings;
CREATE TEMP TABLE toy_bookings AS
SELECT * FROM (VALUES
    ('A', 'City',   10, 'Direct',    'None',       1, 1, 0),
    ('B', 'City',  120, 'TA/TO',     'None',       0, 0, 1),
    ('C', 'Resort', 90, 'TA/TO',     'None',       0, 2, 0),
    ('D', 'Resort',  5, 'Direct',    'Nonrefund',  0, 0, 0),
    ('E', 'City',  200, 'TA/TO',     'None',       0, 0, 1),
    ('F', 'City',   40, 'Corporate', 'None',       1, 0, 0),
    ('G', 'Resort', 60, 'Direct',    'Refundable', 0, 1, 1),
    ('H', 'Resort', 15, 'TA/TO',     'None',       0, 0, 0)
) AS t(
    booking_id, hotel, lead_time, channel, deposit,
    repeat_guest, requests, is_canceled
);

-- Question: What share of eligible bookings canceled overall?
SELECT
    count(*) AS bookings,
    sum(is_canceled) AS cancellations,
    avg(is_canceled::numeric) AS cancellation_rate
FROM toy_bookings;
```

The fictional output is bookings = 8, cancellations = 3, rate = 0.375. A dashboard can format 0.375 as 37.5%; do not multiply by 100 twice.

Read the calculation line by line

- `count(*)` counts rows: your booking denominator.
- `sum(is_canceled)` adds the ones: the cancellation numerator.
- `avg(is_canceled::numeric)` averages a validated binary outcome, giving the same rate.
- `FROM` identifies the exact population being analyzed.

Important: SQL averages ignore NULL. If the target contains NULL, the rate denominator can differ from `count(*)`. Validate the target first and explain any exclusion.

Compare hotel types

```
-- Question: How does cancellation rate differ by hotel type?
SELECT
    hotel,
    count(*) AS bookings,
    sum(is_canceled) AS cancellations,
    avg(is_canceled::numeric) AS cancellation_rate
FROM toy_bookings
GROUP BY hotel
ORDER BY hotel;
```

FICTIONAL ANSWER

City has 2 cancellations among 4 bookings: 50%. Resort has 1 among 4: 25%. The overall rate is 3/8 = 37.5%. These tiny groups are a teaching device, not evidence about real hotel performance.

Turn a numeric field into meaningful bands

```
-- Question: How does cancellation rate vary by lead-time band?
WITH banded AS (
    SELECT
        CASE
            WHEN lead_time < 30 THEN '01: under 30 days'
            WHEN lead_time < 90 THEN '02: 30-89 days'
            ELSE '03: 90+ days'
        END AS lead_time_band,
        is_canceled
    FROM toy_bookings
)
SELECT
    lead_time_band,
    count(*) AS bookings,
    sum(is_canceled) AS cancellations,
    avg(is_canceled::numeric) AS cancellation_rate
FROM banded
GROUP BY lead_time_band
ORDER BY lead_time_band;
```

`CASE` creates the labels; `WITH` names an intermediate result. The bands are illustrative choices, not discovered optimum thresholds. Invalid or missing lead times must be handled before this query, or explicitly placed in an Unknown group.

The full query checklist

Suggested file	Question it answers	Important detail
`01_overall.sql`	What is the overall cancellation rate?	Show numerator and denominator.
`02_hotel.sql`	How does it differ by hotel type?	Include group size.
`03_lead_time.sql`	How does it vary by booking lead time?	Document non-overlapping bands.
`04_market_segment.sql`	Which commercial segments differ?	Do not confuse this with your custom behavior segments.
`05_distribution_channel.sql`	How do distribution channels compare?	Keep missing/unknown categories visible.
`06_deposit_type.sql`	How do deposit arrangements compare?	Do not infer a deposit policy caused the pattern.
`07_customer_type.sql`	How do customer classifications compare?	Use the actual dictionary definitions.
`08_season.sql`	How does cancellation vary by season?	Define season using scheduled arrival date.

A transparent season definition could be Dec-Feb = winter, Mar-May = spring, Jun-Aug = summer, Sep-Nov = autumn. This is a declared calendar convention, not a claim about hotel-specific demand. Also inspect year-month trends because partial years and changing hotel mix can distort seasonal comparisons.

Try it: one group has 1 cancellation out of 2 bookings; another has 90 out of 100. Is the overall rate the average of 50% and 90%?

Show the worked answer

No. These fictional groups have different sizes. Combine their counts: 91 / 102 = approximately 89.2%, not 70%. Use a weighted calculation or recompute from underlying bookings.

Stop and review: every required dimension has its own commented SQL file, counts reconcile, missing categories are explained, and comparisons do not overclaim causation.

07 / PHASE 3

Segmentation: turn many rows into understandable groups

A segment is a defined group of bookings. It is not automatically a risk prediction. The best starting point here is a small rule set a nontechnical operations colleague can explain.

Use ordered rules so each booking belongs once

The following rules use every requested behavior: lead time, channel, deposit, repeat status, and requests. Rules are applied top to bottom; the first matching rule wins.

```
-- Question: What are the sizes and rates of behavior segments?
WITH segmented AS (
    SELECT
        *,
        CASE
            WHEN repeat_guest = 1
                THEN 'Returning guest'
            WHEN lead_time >= 90
                 AND channel = 'TA/TO'
                 AND deposit = 'None'
                THEN 'Advance agency, no deposit'
            WHEN requests >= 1
                THEN 'Other with requests'
            ELSE 'Other bookings'
        END AS segment
    FROM toy_bookings
)
SELECT
    segment,
    count(*) AS bookings,
    sum(is_canceled) AS cancellations,
    avg(is_canceled::numeric) AS cancellation_rate
FROM segmented
GROUP BY segment
ORDER BY bookings DESC, segment;
```

Fictional segment results from the eight-booking teaching table.

Segment	Members	Size	Canceled	Rate
Returning guest	A, F	2	0	0%
Advance agency, no deposit	B, C, E	3	2	66.7% rounded
Other with requests	G	1	1	100%
Other bookings	D, H	2	0	0%

Notice that A has a request but belongs to Returning guest. C has requests but belongs to Advance agency, no deposit. This is intentional precedence, not an error. Change the order and you change the segment definitions.

> **Do not overread small groups.** G's segment has a fictional 100% rate because it contains only one booking. That does not make it a reliable high-risk category. Always show sizes and consider uncertainty or minimum reporting sizes.

Why not clustering immediately?

Clustering discovers groups based on a similarity definition. That introduces choices about scaling, encoding, distance, and number of clusters. Your prompt deliberately starts with rules so the first version stays understandable. Add clustering only after you explicitly request it.

For later predictive use: define rules and thresholds using business reasoning or earlier training data. Do not inspect final test outcomes, redesign segments around them, and then claim an independent test.

Try it: a returning guest books 150 days in advance through TA/TO with no deposit. Where do they belong in these rules?

Show the answer

Returning guest, because that rule appears first. If you want overlapping segments instead, that is a different design: totals may exceed the number of bookings and must be labeled accordingly.

Stop and review: approve rule order, thresholds, missing-value handling, group sizes, and the difference between descriptive groups and individual predictions.

08 / PHASE 4

A cancellation model without cheating

A model learns relationships from historical features and known outcomes. For a new booking, it produces a cancellation probability. That probability is an estimate, not a guarantee or a judgment about the guest.

Features and target

Features are inputs, such as lead time and channel. The **target** is the value being predicted: `is_canceled`. Never include the target itself among the inputs.

Why logistic regression first?

Logistic regression combines numeric inputs and encoded categories into a score and converts it to a probability between zero and one. It is a strong baseline because it is relatively simple to explain and inspect.

FICTIONAL INTERPRETATION EXAMPLE

If a fitted feature has an odds ratio of 1.5, it is associated with 1.5 times the cancellation odds for the stated feature change, holding the other modeled inputs fixed. That does **not** mean a 50-percentage-point increase in probability, and it does not establish cause. Feature scaling and category reference levels must be stated when explaining coefficients.

A shallow decision tree can be the one tree-based comparator. It learns branching rules and may capture interactions logistic regression misses, but can overfit. Compare on the same features and chronological evaluation population.

Leakage: information from the future

Input	Booking-time suitability	Reason
Lead time recorded at booking	Candidate input.	Can be known when the booking is made.
Final reservation status	Exclude.	Contains or closely reveals the outcome.
Reservation status date	Exclude from features.	Outcome-related timing not available at booking.
Assigned room type, changes, waiting-list duration	Audit carefully; exclude unless timing is defensible.	May reflect events after booking creation.
Requests, deposit, or rate in a final extract	Verify their snapshot timing.	A plausible field name is not proof that its recorded value was available at prediction time.

```
# These describe the eventual outcome and would leak future
# information into a cancellation prediction.
leakage_columns = ["reservation_status", "reservation_status_date"]
model_data = clean_data.drop(columns=leakage_columns)
```

```
# Still choose an explicit audited feature list; dropping two
# known leakage columns does not make every other column safe.
```

Split by time, not by shuffling

ILLUSTRATIVE SPLIT DATES - NOT OPTIMIZED OR EVALUATED

Period	Purpose	Allowed decisions
Earlier period: before April 2017	Training	Learn imputations, encodings, scaling, and model parameters.
Middle period: April-June 2017	Validation	Select settings, threshold, and the preferred model without accessing final test outcomes.
Later period: July 2017 onward	Final test	Evaluate the frozen approach once and report the requested metrics.

In an actual implementation, first inspect date coverage and class counts. Your prompt does not prescribe these dates. The split must contain enough examples of both outcomes to make evaluation meaningful.

> **Important dataset limitation:** scheduled arrival date is not booking creation date. If you split by arrival date, describe it as a later-arrival holdout, not a perfect booking-time backtest. Long-lead future-arrival bookings may have been created before earlier-arrival bookings. A derived booking date (arrival date minus lead time) is only a proxy whose semantics need validation.

Label availability also matters: when simulating a historical deployment, training outcomes must already have been known at that cutoff. A booking created before the cutoff but resolved later is not a usable labeled example at that moment. Outcome dates can inform a carefully documented audit without being model features. If this dataset cannot support a faithful as-of reconstruction, state that limitation explicitly.

Fit preprocessing on training data only

Suppose missing lead time is replaced by a median. Calculate that median from the training set only. Learning it from all records lets future data influence the model. The same rule applies to scaling, category encoding, feature selection, and any learned calibration step.

The following illustrative code assumes `train`, `validation`, and `test` are already cleaned, chronologically separated pandas frames returned from PostgreSQL, and that feature timing has been reviewed. The feature choice is a teaching example, not a timing certification. The actual runnable starter is `src\model.py`.

```
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder, StandardScaler

numeric = ["lead_time", "is_repeated_guest"]
```

```
categorical = ["hotel", "distribution_channel", "deposit_type"]
features = numeric + categorical

preprocess = ColumnTransformer([
    ("numeric", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), numeric),
    ("category", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("encode", OneHotEncoder(handle_unknown="ignore")),
    ]), categorical),
])

model = Pipeline([
    ("preprocess", preprocess),
    ("classifier", LogisticRegression(max_iter=1000)),
])

# Only the training partition teaches the pipeline.
model.fit(train[features], train["is_canceled"])
validation_risk = model.predict_proba(validation[features])[:, 1]

# Choose settings and a threshold using earlier validation data.
# Freeze them before inspecting final test results.
test_risk = model.predict_proba(test[features])[:, 1]
```

Encoding turns categories into machine-readable indicator columns. Scaling puts numeric inputs on comparable scales. `handle_unknown="ignore"` allows a later unseen category, but you should still monitor and report category drift rather than treat it as irrelevant.

A fair comparison

Use a fresh preprocessing pipeline for a shallow decision tree, tune it on earlier validation data, and keep the final test untouched. Compare both models using the same final holdout. Include a simple baseline, such as the training cancellation rate assigned to every booking, so you know whether the extra complexity adds value.

Do not repeatedly adjust the model after seeing final test scores. If you do, that period becomes development data and a new independent test period is needed.

Stop and review: approve the prediction moment, feature availability, date semantics, label-availability limitations, baseline, split, and final test report before producing dashboard risk summaries.

09 / MODEL EVALUATION IN PLAIN ENGLISH

What the scores actually mean

Precision and recall depend on a threshold: the risk level at which you flag a booking. ROC-AUC evaluates ranking across thresholds. Calibration checks whether probabilities are believable. None of these alone tells you whether an intervention helps the business.

Start with a confusion matrix

FICTIONAL TEST SET OF 20 BOOKINGS - SEPARATE FROM THE EIGHT-BOOKING SQL EXAMPLE

	Actually canceled	Actually not canceled
Flagged as likely to cancel	6 true positives (TP)	2 false positives (FP)
Not flagged	3 false negatives (FN)	9 true negatives (TN)

Precision: among the 8 flagged bookings, 6 canceled. Precision = 6 / (6 + 2) = **75%**. This helps describe how many alerts are correct.

Recall: among the 9 actual cancellations, the model caught 6. Recall = 6 / (6 + 3) = **66.7% rounded**. This describes how many cancellations were detected.

False positives create unnecessary outreach. False negatives miss opportunities to act. A lower threshold generally flags more bookings and often raises recall at the cost of precision, but inspect the actual validation tradeoff.

ROC-AUC: can the model rank risk?

Imagine taking one canceled booking and one non-canceled booking. Does the model give the canceled one the higher score? ROC-AUC summarizes this ranking ability, counting a tied pair as half correct.

FICTIONAL RANKING EXAMPLE

Canceled bookings have risks 0.9 and 0.6. Non-canceled bookings have risks 0.7 and 0.2. There are four cross-outcome pairs. The model ranks three correctly: 0.9 beats both non-canceled scores, and 0.6 beats 0.2 but not 0.7. AUC = 3 / 4 = **0.75**.

A constant-risk baseline has AUC 0.5 when both classes are present. AUC 1.0 is perfect ranking. AUC is **not** accuracy, not the percentage of cancellations prevented, and not a calibration score. AUC is undefined when the evaluated set contains only one outcome class.

Calibration: can you trust a probability?

FICTIONAL CALIBRATION EXAMPLE

Suppose 10 bookings each receive a predicted risk of 0.8, but only 4 cancel. Their mean predicted risk is 80%, while the observed rate is 40%. The model is overconfident in this group. A model can rank cases well and still give misleading probabilities.

For a calibration check, group final test predictions into risk bins and report each bin's count, average predicted risk, and observed cancellation rate. Small bins have substantial uncertainty. If you later fit a probability calibrator, fit it using earlier data, not the final test.

What final test evaluation code looks like

This example continues the variables from the modeling chapter. The threshold is illustrative; it must be selected and frozen using earlier validation data.

```
import pandas as pd
from sklearn.calibration import calibration_curve
from sklearn.metrics import (
    precision_score, recall_score, roc_auc_score,
)

y_test = test["is_canceled"]
threshold = 0.50  # Illustrative; select before final test evaluation.
test_flag = test_risk >= threshold

if y_test.nunique() != 2:
    raise ValueError("Final test data must contain both outcome classes.")
if not test_flag.any():
    raise ValueError("No test alerts; precision is undefined. Report this.")

scores = {
    "precision": precision_score(y_test, test_flag),
    "recall": recall_score(y_test, test_flag),
    "roc_auc": roc_auc_score(y_test, test_risk),
}
observed, predicted = calibration_curve(
    y_test, test_risk, n_bins=5, strategy="quantile"
)

# Group counts should also accompany the calibration display.
test_predictions = pd.DataFrame({
    "actual": y_test.to_numpy(),
    "risk": test_risk,
})
```

Quantile bins aim for similar group sizes; identical risk values can reduce the number of nonempty bins. Preserve the realized bin definitions and counts. Do not replace undefined metrics with a flattering number or silently tune against the test set.

Why not report only accuracy?

FICTIONAL IMBALANCE EXAMPLE

If only 10 out of 100 bookings cancel, a model that says "not canceled" for everyone has 90% accuracy and catches no cancellations. High accuracy can coexist with zero operational usefulness.

Try it: which metric matters if the team can afford only a small number of reminder calls, and which matters if missing cancellations is especially costly?

Show the explanation

Precision is particularly relevant to limiting wasted calls, while recall describes cancellations missed. In practice, choose a threshold using earlier validation data, team capacity, intervention cost, and benefit assumptions. A useful additional capacity metric is precision among the top-ranked bookings, but it does not replace the required metrics.

10 / PHASE 5

Dashboard data: export answers, not confusion

Your task here is to prepare clean CSV summaries. You will build the Power BI or Tableau dashboard yourself. Define the **grain** of each file: what one row represents.

File	One row represents	Useful columns
`cancellation_trends.csv`	One scheduled arrival month and hotel.	arrival_month, hotel, bookings, cancellations, cancellation_rate
`segment_comparison.csv`	One behavior segment.	segment, bookings, cancellations, cancellation_rate
`risk_distribution.csv`	One predicted-risk bin in the final test population.	risk_band, bookings, mean_predicted_risk, observed_cancellation_rate, population

Calculate a trend in PostgreSQL

```
-- Question: How do cancellation rates vary by arrival month and hotel?
SELECT
        date_trunc('month', arrival_date)::DATE AS arrival_month,
        hotel,
        count(*) AS bookings,
        sum(is_canceled) AS cancellations,
        avg(is_canceled::numeric) AS cancellation_rate
    FROM hotel_booking.bookings_clean
    GROUP BY arrival_month, hotel
    ORDER BY arrival_month, hotel;
```

The actual export command runs this kind of PostgreSQL query and writes the returned rows locally with Python. You do not need PostgreSQL server-side filesystem permissions. The month is the scheduled stay month, not the month someone clicked Cancel. A cancellation-event timeline needs a separate, carefully defined query using outcome timing.

Common dashboard mistakes

- **Summing rates:** use total cancellations / total bookings for the filtered population. Do not sum percentages or average differently sized groups without weights.
- **Joining incompatible grains:** joining one row per segment onto one row per month can multiply rows. Keep separate fact tables or design a proper model.
- **Showing mixed populations:** historical SQL totals may cover all clean records, while risk scores cover only the later test period. Label those populations visibly.
- **Hiding zero-booking months:** if adding a calendar to show gaps, use zero counts but leave cancellation rate undefined when the denominator is zero.
- **Calling risk a certainty:** a high-risk band still includes bookings that do not cancel.

Store rates as decimals between 0 and 1 and format them as percentages in the visualization. Document bin boundaries, segment rules, refresh date, and source population. Export aggregate data rather than unnecessary record-level details.

Stop and review: inspect CSV headers, row grain, population labels, rate formatting, and reconciliation to SQL/model outputs before Phase 6.

11 / PHASE 6

Write a project someone else can understand

The README is a small case study and a runbook. It should make the project understandable to both a hiring manager and a technical reviewer.

Section	What to explain	Safe example language
Question	The decision and analysis population.	"Investigate historical booking cancellations and evaluate later-period risk prediction."
Data	Source, unit of observation, coverage, cleaning reference.	"Booking-level data; cleaning and exclusions are documented in DATA.md."
Method	SQL analysis, ordered segments, feature audit, chronological evaluation.	"Compared logistic regression and one tree-based model using a documented temporal holdout."
Results	Verified outputs only, with the right populations.	"Overall cancellation rate: [RESULT: overall cancellation rate]. Test ROC-AUC: [RESULT: test ROC-AUC]."
Limitations	What conclusions cannot safely generalize.	"The model is not validated on other markets."
Reproduce	Download instructions, environment, commands, expected outputs.	"Place the CSV in data/raw/, install the tested dependencies, then run the documented pipeline."

Required limitations, and what they mean

Two hotels in one country: this is a narrow setting, not a representative sample of the global hotel industry.

Data from 2015-2017: booking channels, traveler behavior, and cancellation policies may have changed. Historical performance is not evidence of present-day deployment performance.

Not validated on other markets: even a good internal holdout score does not establish reliability for another country, hotel platform, or business model.

Also explain relevant snapshot/feature timing uncertainty, partial-year coverage, observational rather than causal evidence, and limitations in reconstructing when outcomes became available.

Your recommendation file stays yours

```
# Context

# Findings

# Recommendation

# Risks
```

The original instruction says headings only. An assistant should not fill in these sections on your behalf. To prepare, ask yourself: What decision is at stake? Which verified findings support it? What limited pilot would you propose? What could go wrong? Your answers go into that file when you choose to write it.

Placeholders prevent accidental fabrication

Before results exist, write `[RESULT: final test recall]`, not "approximately 80% recall." A plausible number is still invented if it was not calculated. Teaching examples in this guide must never be copied into the project's results section.

Stop and review: check every factual result against a saved output; confirm limitations and reproduction steps; keep recommendation.md headings-only until you write its content.

12 / THROUGHOUT THE PROJECT

Small engineering habits that make analysis credible

Type hints: make intent visible

```
"""Validate booking labels before computing cancellation summaries."""

def cancellation_rate(labels: list[int]) -> float:
    if not labels:
        raise ValueError("At least one booking is required.")
    if any(label not in (0, 1) for label in labels):
        raise ValueError("Cancellation labels must be 0 or 1.")
    return sum(labels) / len(labels)
```

`list[int]` describes the expected input; `-> float` describes the output. Type hints aid readers and tooling but do not validate data automatically. The checks explicitly reject invalid inputs instead of returning a misleading zero.

This tiny function is for learning Python and tests; the project's main cancellation aggregations still belong in SQL. The opening docstring explains *why* the module exists, not merely "this file contains functions."

pytest: turn assumptions into checks

```
import pytest
from src.quality import cancellation_rate

def test_known_fictional_example() -> None:
    assert cancellation_rate([0, 1, 0, 1]) == pytest.approx(0.5)

def test_invalid_label_is_rejected() -> None:
    with pytest.raises(ValueError, match="0 or 1"):
        cancellation_rate([0, 2])

def test_empty_population_is_rejected() -> None:
    with pytest.raises(ValueError, match="At least one"):
        cancellation_rate([])
```

These examples assume the function has been saved in `src\quality.py`. They illustrate tests; that module is not created by this guide.

What actual data-quality tests should cover

- Required columns exist and conversions are audited.
- The analytical target is non-null and binary.
- Retained records obey approved guest-count and date rules.

- Confirmed zero guests are excluded if that policy was approved.
- Raw, retained, and excluded populations reconcile without double counting.
- Each booking receives exactly one segment under exclusive rules.
- Training, validation, and test periods are ordered and disjoint.
- Known leakage columns are not model features.
- Exported cancellation counts never exceed their booking counts.

A test that passes does not prove all data are correct. It proves that the specific assertion holds for the tested input. Test the cleaning transformations as well as the final table: a bad cleaning rule can produce a neat-looking table.

Dependencies and reproducibility

A requirements file should describe the tested Python packages. Record the Python version, dataset source/version or file checksum, execution order, output locations, and any random seed used by the comparator. A seed helps reproduce stochastic behavior; it does not fix a bad split.

Small commits

A commit is a focused saved change in Git. Useful commit subjects might be "Add raw import and schema audit," "Document zero-guest policy," or "Add cancellation queries by channel." Avoid one giant commit that hides every decision. Never commit secrets, local environments, or raw data by accident.

What a phase handoff should contain

State what changed, where the outputs are, which assumptions need approval, and any known limitations. Then stop. "Work in phases" is a collaboration rule, not just a set of section headings.

13 / CONNECT THE PIECES

Follow one booking all the way through

FICTIONAL BOOKING B FROM OUR TEACHING TABLE

1. Raw record

B is a City booking, made 120 days ahead, through TA/TO, with no deposit, no repeat-guest flag, no special requests, and a cancellation outcome of 1.

2. Quality layer

Before B enters the analysis, validate its label, guest fields, numeric fields, and constructed arrival date. Retain or exclude it according to the agreed policies, not because its outcome is inconvenient.

3. SQL summaries

B contributes one booking and one cancellation to the overall rate, the City group, the TA/TO channel, and the appropriate lead-time band. Groupings answer different questions; adding all those group totals together would count the same booking several times.

4. Behavior segment

B is not a repeat guest. It then satisfies the advance-agency/no-deposit rule and joins B, C, and E. That segment's fictional rate is 2/3, but it is not automatically B's individual model probability.

5. Model input

If B falls in the final test period, the model sees only its audited, allowable features. It must not see B's cancellation label or final status when predicting. Its actual outcome is used afterward to evaluate the prediction.

6. Risk score

Suppose, purely for explanation, B receives a risk of 0.72. This does not mean it will "72% cancel" as a single event. It means the model estimates a 72% probability; among many comparably scored bookings, calibration asks whether the observed frequency is similar.

7. Dashboard

B contributes to the historical arrival-month summary. If scored as part of the final test, it also contributes to the corresponding test risk bin. The dashboard should not imply those two files cover identical populations.

8. Business action

An operations team could consider a reminder pilot for a defined population. Before claiming value, define the intervention, cost, success measure, guest safeguards, and comparison group. Model evaluation alone cannot establish that the reminder works.

> **The whole chain:** business decision → raw records → documented clean population → SQL patterns → understandable groups → leakage-aware prediction → honest evaluation → dashboard evidence → a testable business recommendation.

14 / TELL THE TRUTH, CLEARLY

Turn completed work into a resume entry

The strongest resume bullet says what you did, how you did it, and what you can substantiate. Do not describe planned work as completed work. Reading this guide alone does not establish the accomplishments below.

If the project is still in progress

Hotel Booking Cancellation Analysis | Personal project, in progress

Developing a SQL-first hotel booking analysis workflow with PostgreSQL, DBeaver, and Python, focusing on documented data cleaning, interpretable booking segments, and chronological cancellation-risk evaluation.

Use this only once development is genuinely underway, and remove any part you have not started. If you have only planned the project, do not list it as an implemented portfolio project yet.

After the work is completed

Hotel Booking Cancellation Analysis & Risk Modeling

Python, SQL, PostgreSQL, DBeaver, scikit-learn, pytest

- Analyzed hotel booking cancellations using PostgreSQL across hotel types, lead times, market segments, distribution channels, deposit types, customer types, and seasons.
- Developed rule-based booking segments and compared logistic regression with a tree-based model using chronological evaluation and explicit leakage controls.
- Documented data-cleaning decisions, implemented automated data-quality checks, and exported dashboard-ready CSV summaries for trend, segment, and risk analysis.

Add verified metrics only after evaluation

"Evaluated cancellation risk on a later-period holdout of `[RESULT: test booking count]` bookings, achieving ROC-AUC `[RESULT: test ROC-AUC]` and recall `[RESULT: test recall]` at a validation-selected threshold."

Replace every placeholder before putting this on your resume. Include sample size, period, and threshold context in the repository so the score can be understood. Do not copy fictional numbers from this guide.

> **Claims to avoid:** "reduced cancellations," "increased hotel revenue," "production-ready," and "built a Power BI dashboard" unless those specific outcomes happened and you can support them. Preparing CSVs is not the same as building a dashboard.

Practice explaining it in an interview

Question	A useful answer structure
Why PostgreSQL and DBeaver?	"PostgreSQL runs the analytical SQL, while DBeaver provides an interface I already use to inspect and explain the queries."
Why not a random split?	"The task predicts later bookings, so I held out a later period and documented which date I used and its limitations."
How did you avoid leakage?	"I removed final status fields, audited feature timing, and fit preprocessing only on earlier training data."
Why start with logistic regression?	"It is an interpretable baseline. I compared it with one tree-based model rather than assuming complexity would help."
What was the biggest limitation?	"Only two hotels in one country, older data, no external-market validation, and limitations reconstructing booking-time snapshots."
Did the project reduce cancellations?	"It analyzed and predicted cancellations. Measuring a reduction would require deploying and evaluating an intervention."

15 / KEEP THIS PAGE NEARBY

A compact glossary and your next practical step

Term	Meaning in this project
Population	The exact set of bookings included in a calculation.
Denominator	The total eligible bookings under a rate.
Grain	What one row represents.
NULL	A missing/unknown database value; not automatically zero or an empty category.
Imputation	A documented method for filling missing model inputs.
One-hot encoding	Representing categories using separate indicator features.
Feature	An input used to make a prediction.
Target / label	The outcome the model learns to predict.
Leakage	Using information not legitimately available at prediction time or allowing evaluation information to influence training.
Temporal holdout	A later period reserved for evaluating a model trained on earlier data.
Overfitting	Learning patterns that work well on development data but do not generalize.
Threshold	The probability cutoff used to turn risks into alerts.
Calibration	Agreement between predicted probabilities and observed frequencies.
Drift	Changes in inputs, outcomes, or their relationships over time.
Baseline	A simple reference approach that a more complex model should justify improving on.
Reproducibility	Another person can follow the documented inputs and commands to obtain consistent outputs.

Start with Phase 1 when you are ready

Starter code for every phase is available, but you still begin by downloading the CSV, configuring your database, and reviewing the actual Phase 1 quality counts. Continue manually, one phase at a time, using the next chapter and README.md.

This guide does not download your data, fabricate findings, run the real-data workflow, write your recommendation content, or push anything to GitHub.

Official references for further reading

- Hotel booking demand datasets: original dataset publication - consult the data dictionary for field semantics.
- PostgreSQL COPY documentation - importing/exporting records and client/server file distinctions.

- PostgreSQL SELECT documentation - SQL query structure.
- scikit-learn common pitfalls - leakage and inconsistent preprocessing.
- scikit-learn probability calibration - understanding risk estimates.
- pytest getting started - writing and running checks.

> **What you should now be able to explain:** why a rate needs a clear denominator; why missing does not mean zero; why ordered rules matter; why future information invalidates predictions; why precision, recall, ranking, and calibration answer different questions; and why honest limitations strengthen a portfolio project.

16 / YOUR MANUAL EXECUTION WORKBOOK

All phases are prepared. Run them later, one at a time.

This chapter is the print-friendly command companion to **README.md**. Nothing here runs automatically. No real-data findings, recommendation content, commit, or push has been produced for you.

> **Before starting:** use the project worktree, configure your dedicated PostgreSQL database and `.env`, download the CSV, and review DATA.md. Generated CSV/model bundles and credentials stay ignored by Git. Never share raw predictions or passwords accidentally.

Phase 1: import, then inspect

```
.\.venv\Scripts\python.exe -m src.load_data
```

In DBeaver, refresh `Schemas > hotel_booking`. Inspect `bookings_raw`, `bookings_quality`, the `bookings_clean` view, and `load_audit`. Run `sql\phase1\03_quality_report.sql`.

Stop: confirm raw = retained + excluded; investigate reasons and warnings. Review missing children, confirmed zero guests, invalid dates, and identical-row retention. Use `--replace` only when deliberately refreshing the imported snapshot.

Phase 2: answer the SQL questions

```
.\.venv\Scripts\python.exe -m src.analyze --phase 2
```

Alternatively, open individual files in `sql\phase2\` in DBeaver and select your PostgreSQL connection. The runner saves the same SQL results as CSV files and records the source snapshot in `data\processed\phase2\manifest.json`.

Files	What you learn
01 overall / 02 hotel	Population baseline and hotel-type comparison.
03 lead time / 04 market segment	Advance-booking bands and commercial groups.
05 distribution channel / 06 deposit type	Booking routes and deposit arrangements.
07 customer type / 08 season	Customer classes and arrival seasons within each year.

Stop: verify counts and denominators. Missing categories remain blank in CSV and must not disappear from your dashboard. Aggregate rates using summed counts, not an unweighted average. Partial-year data can distort seasonal comparisons.

Phase 3: build and explain behavior groups

```
.\.venv\Scripts\python.exe -m src.analyze --phase 3
```

This defines `hotel_booking.booking_segments` and exports `data\processed\phase3\02_segment_summary.csv`. In DBeaver, run `01_behavior_segments.sql` before `02_segment_summary.sql` if working manually.

1. Repeat guest → Returning guest.
2. Otherwise, lead time at least 90 days + TA/TO + No Deposit → Advance agency, no deposit.
3. Otherwise, at least one request → Other with requests.
4. Otherwise → Other bookings.

Stop: explain the first-match rule. Missing channel/deposit does not satisfy the advance rule. Sizes must sum to the eligible population; rates from tiny segments should not be treated as reliable predictions. No clustering is added.

Phase 4: choose your dates before fitting

First run `sql\phase4\02_date_coverage.sql` in DBeaver. Inspect monthly counts of each outcome, choose defensible boundaries, and do not optimize the test period for a flattering score.

ILLUSTRATIVE DATES ONLY - CHOOSE AFTER COVERAGE REVIEW

```
.\.venv\Scripts\python.exe -m src.model `
  --validation-start 2017-04-01 `
  --test-start 2017-07-01 `
  --acknowledge-snapshot-limitations
```

Earlier arrivals train the model; the middle period selects thresholds; later arrivals form the final test. Each partition must have both outcomes. This is not a random split. The acknowledgment makes the final-extract timing limitation explicit; it does not certify a booking-time snapshot.

> **A later-arrival holdout is not a booking-time backtest.** Booking creation order can differ from arrival order. Some inputs may have been updated after booking, and training labels may not have been known at a simulated historical cutoff. The scaffold does not reconstruct these histories and is not deployment validation.

Both models explicitly exclude final status and status date, use an allowlist, and fit preprocessing only on training. Logistic regression is the prespecified primary; one shallow tree is the comparator. A constant training-rate baseline provides context.

Each model's threshold is chosen using earlier validation F1 on a fixed grid from 0.1 to 0.9, with deterministic tie-breaking. This is a transparent teaching default, not a cost- or capacity-optimized operations policy. No threshold is selected from final test outcomes.

File in data\processed\phase4\	How to read it
test_metrics.json	Test-only precision, recall, ROC-AUC, Brier score, confusion counts, and frozen thresholds for both models and baseline.
test_calibration.csv	Test-bin sizes, average predictions, and observed cancellation rates.
test_predictions.csv	Local-only final-test probabilities from the prespecified logistic primary.
logistic_regression_features.csv	Coefficients on standardized numeric inputs and one-hot categories; not causal effects.
decision_tree_features.csv	Feature importances; they do not supply a direction or prove cause.
manifest.json	Cutoffs, source snapshot, input list, software versions, checksum, and limitations.

Calibration uses fixed tenths; empty bins have count zero and blank rates, and the final bin includes probability 1. If no bookings are flagged, precision is recorded as undefined with an explicit note, not replaced by a flattering score.

Stop: compare with the baseline, explain the calibration, and retain all required limitations. Do not select a primary by repeatedly inspecting the final test. Saving another folder does not create an independent holdout.

Phase 5: export dashboard tables

```
.\.venv\Scripts\python.exe -m src.export_dashboard
```

The exporter requires the Phase 3 segment view and a reviewed Phase 4 model run. It rejects a prediction-file checksum mismatch or a different imported database snapshot, rather than silently mixing populations.

File in data\exports\phase5\	Population and grain
cancellation_trends.csv	All eligible bookings, one row per arrival month and hotel.
segment_comparison.csv	All eligible bookings, one row per behavior segment.
risk_distribution.csv	Final-test later arrivals only, one row per risk band.

The SQL risk display bands are Low [0, 0.3), Medium [0.3, 0.7), and High [0.7, 1]. They are presentation choices, not intervention rules. Empty bins remain visible. A manifest records source and population definitions.

Stop: import the CSVs separately in Power BI or Tableau. Do not create a join that multiplies monthly rows by segment rows. Format decimal rates as percentages and clearly label final-test risk separately from historical totals. You build the dashboard yourself.

Phase 6: write only what you can support

Fill the README's result slots from verified outputs. Explain question, data, method, results, limitations, and reproduction steps. Write your own recommendation in the headings-only `recommendation.md` template: Context, Findings, Recommendation, Risks.

Result to verify later	Placeholder
Overall cancellation rate	[RESULT: overall cancellation rate]
Primary model precision	[RESULT: logistic test precision]
Primary model recall	[RESULT: logistic test recall]
Primary model ROC-AUC	[RESULT: logistic test ROC-AUC]
Calibration interpretation	[RESULT: test calibration interpretation]

Required limitations: two hotels in one country; data from 2015-2017; the model is not validated on other markets. Also state feature/label timing uncertainty, partial-year coverage, lack of causal evidence, and unmeasured operational impact.

Reruns and checks

Later-phase output folders are write-once. For deliberate reruns, add a new `--output` path. If changing the model folder, pass that existing folder to the exporter through `--model-run`. Do not delete earlier outputs just to hide a disappointing result.

```
.\.venv\Scripts\python.exe -m pytest -m "not integration"
```

For SQL-backed checks, use a separate disposable PostgreSQL database and the explicit test configuration from README.md. Tests use fictional records; their outputs are not your dataset findings.

> **Publication is a separate step.** Nothing has been pushed. Before a later commit or push, review secrets, ignored data, licensing, result placeholders, and which learning documents you want to share. Use small focused commits when you choose to commit.